

Sign Language Interpreter

Complete Technical Architecture, Pipeline Specification & Viva Examination Defense Guide

Real-Time Computer Vision

MediaPipe Tasks HandLandmarker

FastAPI + Async WebSockets

Multithreaded Architecture

Decoupled ML Interface

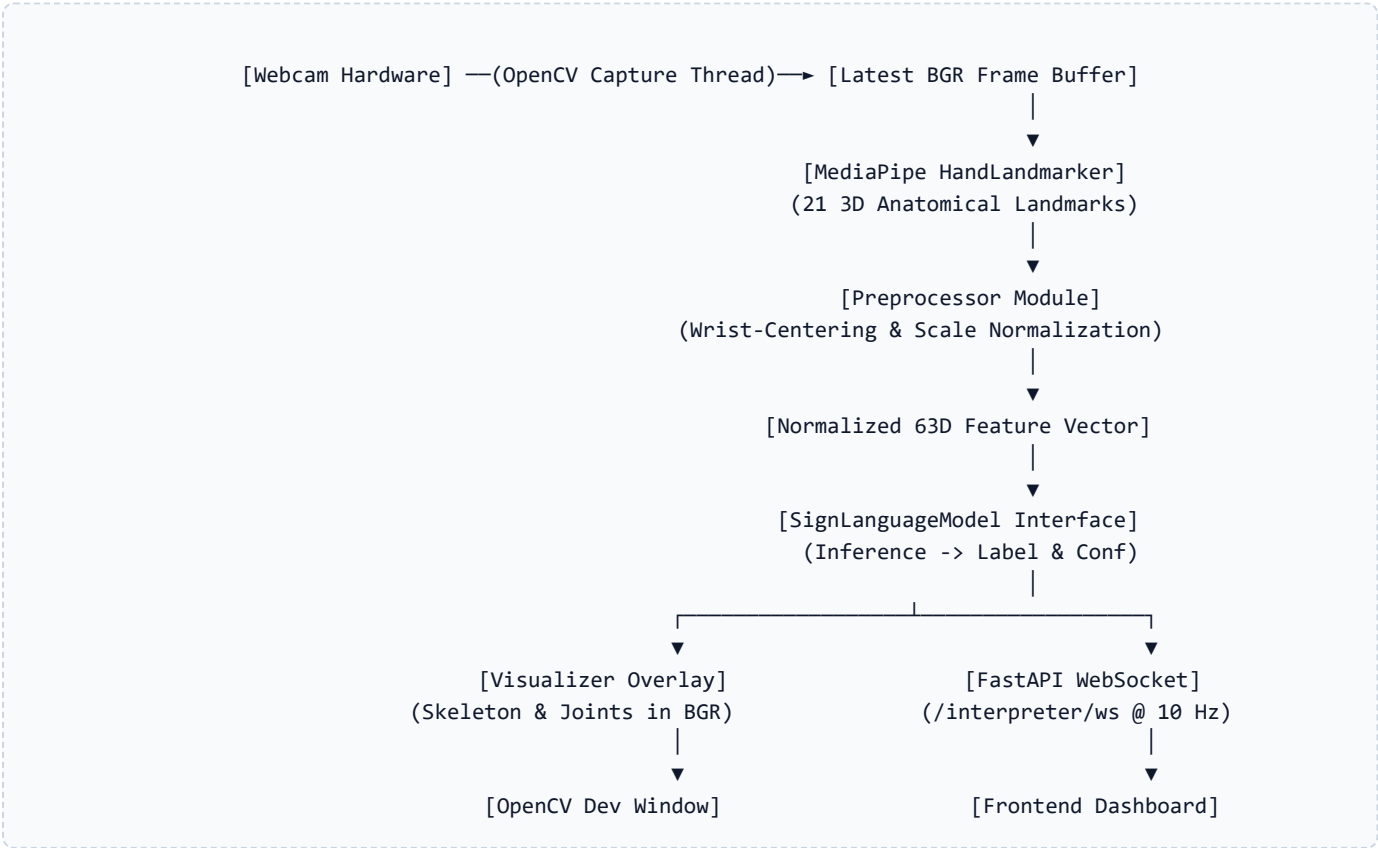
Vanilla JS Dashboard

1. Executive Summary & Core Objective

This project implements a modular, real-time computer vision system that translates American Sign Language (ASL) and hand gestures into text.

Instead of feeding raw webcam frames directly into an end-to-end Convolutional Neural Network (which is computationally expensive, sensitive to lighting, and biased by background clutter or skin tone), the system uses a **two-stage decoupled architecture**:

- 1. **Spatial Keypoint Extraction:** Uses Google's MediaPipe Tasks API to detect hands and extract 21 3D anatomical landmarks per hand.
- 2. **Geometric Normalization:** Centers coordinates on the wrist (translation invariance) and normalizes by palm scale (scale/depth invariance) to produce a 63-dimensional feature vector.
- 3. **Modular Inference & Real-Time Broadcast:** Passes features into an abstract `SignLanguageModel` interface and pushes predictions and telemetry to a responsive web dashboard via WebSockets at 10 Hz.



2. Complete Technology Stack

Layer / Component	Technology	Version	Key Responsibility
Core Runtime	Python	≥ 3.10	Execution runtime for backend server, CV processing, and ML modeling.
Camera & Image Processing	OpenCV (<code>opencv-python</code>)	≥ 4.9.0	Frame capture via <code>cv2.VideoCapture</code> , BGR-to-RGB conversion, skeleton line drawing, local dev GUI.
Hand Landmark Detection	Google MediaPipe (Tasks API)	≥ 0.10.14	Detects 21 3D landmarks (x, y, z) per hand and handedness (Left/Right) via <code>hand_landmarker.task</code> .
Numerical Computing	NumPy	≥ 1.26.0	Wrist subtraction, Euclidean vector norm calculations, array flattening into <code>(63,)</code> vector.
Backend Web Framework	FastAPI	≥ 0.111.0	Asynchronous REST endpoints (<code>/start</code> , <code>/stop</code> , <code>/status</code>) and WebSocket protocol handling.
ASGI Web Server	Uvicorn	≥ 0.29.0	High-speed production ASGI server running the FastAPI event loop.
Settings & Validation	Pydantic & Pydantic-Settings	≥ 2.7.0	Type-safe configuration loaded from <code>.env</code> file; schema validation for HTTP responses.
Real-Time Transport	WebSockets (<code>websockets</code>)	≥ 12.0	Full-duplex persistent TCP socket pushing live telemetry JSON (~10 updates/sec).
Image-based ML (Alternative)	TensorFlow / Keras	2.x	Found in <code>mlcode.py</code> : standalone CNN trained on 224x224 cropped hand images.
Frontend Client	Vanilla HTML5 / CSS3 / ES6+ JS	Modern	Zero-build responsive web dashboard ("SIGNAL") with status LEDs, confidence bar, and CSV export.
Testing Framework	pytest & HTTPX	≥ 8.2.0	Unit and integration tests with mocked camera and detection hardware (no camera needed in CI).

3. System Architecture & Module Breakdown

Module Path	Class / Entry Point	Responsibilities & Architectural Role
<code>backend/camera/camera.py</code>	<code>Camera</code>	Thread-safe capture loop. Runs <code>VideoCapture</code> on a background thread so I/O never blocks the pipeline. Provides <code>read()</code> .
<code>backend/hand_detection/detector.py</code>	<code>HandDetector</code>	Wraps MediaPipe Tasks <code>HandLandmarker</code> . Auto-downloads <code>hand_landmarker.task</code> if missing. Pure image → landmarks.
<code>backend/preprocessing/preprocessor.py</code>	<code>Preprocessor</code>	Subtracts wrist landmark coordinates (wrist-centering) and scales by wrist-to-middle-MCP Euclidean distance.
<code>backend/model/base_model.py</code>	<code>SignLanguageModel (ABC)</code>	Abstract base class defining <code>predict(features) -> Prediction</code> . Decouples ML implementation from the rest of the app.
<code>backend/model/mock_model.py</code>	<code>MockSignLanguageModel</code>	Deterministic mock predictor for development and testing prior to model training.
<code>backend/visualization/visualizer.py</code>	<code>Visualizer</code>	Renders the 21-joint skeleton connections and live status/confidence text overlay onto OpenCV BGR frames.
<code>backend/pipeline/interpreter.py</code>	<code>Interpreter</code>	Central coordinator. Runs the camera → detector → preprocessor → model loop on a background thread paced at 30 FPS.
<code>backend/api/routes.py</code>	<code>APIRouter</code>	REST endpoints: <code>POST /interpreter/start</code> , <code>POST /interpreter/stop</code> , <code>GET /interpreter/status</code> .
<code>backend/api/websocket.py</code>	<code>interpreter_status_ws</code>	WebSocket endpoint at <code>/interpreter/ws</code> streaming live status JSON to client at 10 Hz.
<code>backend/camera.py</code>	<code>collect_data.py</code>	Standalone CLI utility that prompts for a sign label and appends 63D landmark vectors to <code>data/landmarks.csv</code> .

4. Mathematical Feature Preprocessing

The feature engineering in `preprocessor.py` is the foundational reason the system performs reliably across different users and cameras:

1. Raw MediaPipe Landmark Coordinates:

MediaPipe outputs 21 landmarks normalized to image dimensions: $\mathbf{P}_i = (x_i, y_i, z_i) \quad \text{for } i \in [0, 20]$.

2. Translation Invariance (Wrist Centering):

The wrist is landmark index 0 ($\mathbf{P}_0$). We subtract the wrist position from all 21 points:

$$\mathbf{P}'_i = \mathbf{P}_i - \mathbf{P}_0, \quad \text{for all } i \in [0, 20]$$

Impact: The wrist always sits at $(0, 0, 0)$. Gestures are identical whether performed at the top, bottom, or corner of the screen.

3. Scale Invariance (Depth Normalization):

Landmark 9 is the middle finger metacarpophalangeal (MCP) knuckle. We compute the Euclidean distance from wrist to knuckle:

$$s = \|\mathbf{P}'_9\|_2 = \sqrt{(x'_9)^2 + (y'_9)^2 + (z'_9)^2}$$

All centered coordinates are then divided by this reference scale:

$$\mathbf{P}''_i = \frac{\mathbf{P}'_i}{s}$$

Impact: Moving your hand closer to or further from the webcam changes absolute pixel size, but relative finger geometry remains constant.

4. Feature Vector Flattening:

The normalized (21×3) matrix is flattened into a 1D vector of length **63**: $\mathbf{V} = [x''_0, y''_0, z''_0, x''_1, y''_1, z''_1, \dots, x''_{20}, y''_{20}, z''_{20}]$

5. Concurrency & Performance Architecture

- **Thread Isolation:** OpenCV's `VideoCapture.read()` blocks on webcam hardware I/O. If executed inside FastAPI's event loop, all web requests would hang. In this design, `Camera` runs a dedicated background thread buffering the latest frame under a `threading.Lock`.
- **Paced Processing Loop:** The `Interpreter` runs a separate processing loop in a daemon thread. It measures loop duration and sleeps precisely enough to maintain the target `pipeline_target_fps = 30`, preventing unnecessary 100% CPU spikes.
- **Thread-Safe Status Queries:** The web API never touches the camera or MediaPipe directly. It only inspects a thread-safe `InterpreterStatus` dataclass snapshot, guaranteeing instantaneous sub-millisecond REST/WebSocket responses.

6. Examiner Viva Q&A Cheat Sheet (Mandatory Questions)

Q1: Why use MediaPipe landmarks instead of passing raw webcam images directly into a deep CNN?

Answer: A raw image CNN requires processing $(224 \times 224 \times 3 \approx 150,000)$ pixels per frame. It is computationally heavy, sensitive to skin tone variations, room lighting, and background clutter. MediaPipe extracts the 21 geometric 3D hand landmarks, reducing input size from 150,000 pixels to just **63 coordinate numbers**. A classifier on 63 features runs at 30+ FPS on any low-end CPU without requiring a GPU, preserves user privacy, and is completely immune to background noise.

Q2: How do you achieve translation and scale invariance in hand detection?

Answer: Inside `backend/preprocessing/preprocessor.py`, we perform two geometric operations:

1. **Wrist Centering (Translation):** We subtract the wrist coordinate (landmark 0) from all 21 points, fixing the wrist at `(( 0,0,0 ))`.
2. **Scale Normalization (Distance):** We divide all coordinates by the Euclidean distance between the wrist and the middle finger MCP knuckle (landmark 9). This ensures the coordinates represent proportional finger poses regardless of how close or far the hand is from the lens.

Q3: Why did you use WebSockets instead of normal HTTP polling?

Answer: HTTP polling requires establishing a new TCP handshake and sending full HTTP request/response headers 10 to 30 times every second, wasting bandwidth and introducing network latency. WebSockets maintain a persistent, full-duplex TCP connection established once during the handshake. The server simply pushes lightweight JSON telemetry frames every 100ms with virtually zero overhead.

Q4: How does the system handle "No Hand Detected" scenarios? Does it crash or return random guesses?

Answer: In `HandDetector.detect()`, if no hands are visible, MediaPipe returns empty landmarks. The detector returns a `DetectionResult(hand_detected=False)`. The preprocessor detects this and returns `None`. The model interface handles `None` gracefully by returning `Prediction("No sign detected", confidence=0.0)`. It is treated as an expected system state, never an exception.

Q5: How do you replace the Mock Model with a real trained machine learning model?

Answer: The architecture strictly adheres to the *Dependency Inversion Principle*. All models inherit from `SignLanguageModel(ABC)`. To connect a trained model:

1. Create `backend/model/real_model.py` implementing `predict(features)` to evaluate the 63D vector using Scikit-Learn, PyTorch, or TensorFlow.
2. In `backend/main.py`, inside `build_interpreter()`, change `model = MockSignLanguageModel()` to `model = RealSignLanguageModel(model_path=settings.model_path)`.
3. Zero changes are needed in camera capture, detection, API routes, or frontend code.

Q6: What is the purpose of `mlcode.py` vs `backend/camera.py`?

Answer: `mlcode.py` is a standalone experiment testing an image-based CNN (cropping bounding boxes around hands). In contrast, the production backend uses coordinate-based landmark classification. `backend/camera.py` is our dataset collection tool that streams webcam landmarks directly to `data/landmarks.csv` to train coordinate-based classifiers (e.g. Random Forest or MLP).

Q7: What are future improvements for this architecture?**Answer:**

- **Temporal Dynamic Signs:** Add an LSTM, GRU, or 1D Temporal CNN buffer to classify motion-based gestures (e.g., spelling "J" or phrases) over a sequence of 30 frames.
- **Two-Hand Gestures:** MediaPipe and `preprocessor.process_many()` already support 2 hands; we can concatenate both 63D vectors into a 126D input vector.
- **NLP Sentence Synthesis & Audio:** Pipe recognized signs into an LLM or smoothing algorithm to construct fluent English sentences and output audio speech using the browser's Web Speech API.

7. Execution Guide

```
# 1. Activate Environment & Install Dependencies
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# 2. Run Manual Development Preview (OpenCV Window)
python main.py          # Press 'q' to exit

# 3. Run Production Server (FastAPI + Web Dashboard)
uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
# Open frontend/index.html in your browser or run:
python -m http.server 3000 --directory frontend

# 4. Run Automated Test Suite
pytest                  # Mocks hardware; passes without physical webcam
```